

Firestore Google Sign-In & FCM Integration Documentation (Flutter)

Overview

This documentation explains how **Google Sign-In**, **Firebase Authentication**, **Cloud Firestore**, and **Firestore Cloud Messaging (FCM)** are integrated in your Flutter application.

The flow includes:

- Google authentication using Firebase
- Saving user data in Firestore
- Fetching Firebase ID Token
- Requesting notification permission
- Retrieving FCM token
- Navigation after successful login

Dependencies

```
firebase_core: ^4.3.0
google_sign_in: ^7.2.0
firebase_auth: ^6.1.3
cloud_firestore: ^6.1.1
firebase_messaging: ^16.1.0
```

Google Sign-In Flow

`_signinwithGoogle()` **Method**

This function handles the complete Google Sign-In process from UI level.

Responsibilities

- Show loading indicator
- Trigger Google authentication
- Fetch Firebase ID token
- Fetch FCM token
- Navigate to next screen

google signin method use in ui

```
Futlure<void> _signinwithGoogle() async {
  setState(() {
    _isLoading = true;
  });

  try {
    final userCredential = await GoogleSignInService.signInWithGoogle();

    if (!mounted) return;

    if (userCredential != null) {
      final user = userCredential.user;

      if (user != null) {
        // Fetch Firebase ID token
        final idToken = await user.getIdToken();
        print('Firebase ID Token: $idToken');

        // Token expiration time
        final tokenResult = await user.getIdTokenResult();
        print('Token expires at: ${tokenResult.expirationTime}');

        // Fetch FCM Token
        await getFcmToken();
      }

      // Navigate on success
      NavigationService.navigateTo(Routes.usecommonwidget);
    }
  } catch (e) {
    if (!mounted) return;
    print('Google login failed: $e');
    throw Exception('Google login failed: $e');
  } finally {
    setState(() {
      _isLoading = false;
    });
  }
}
```

Firebase Cloud Messaging (FCM)

getFcmToken() Method

This function handles notification permission and retrieves the FCM token.

Responsibilities

- Request notification permission (important for iOS)
- Generate unique device FCM token

FCM message

```
Future<void> getFcmToken() async {
  FirebaseMessaging messaging = FirebaseMessaging.instance;

  NotificationSettings settings = await messaging.requestPermission(
    alert: true,
    badge: true,
    sound: true,
  );

  if (settings.authorizationStatus == AuthorizationStatus.authorized) {
    String? token = await messaging.getToken();
    print('FCM Token: $token');
  } else {
    print('User declined notification permissions');
  }
}
```

Use Case:

- Save this FCM token in Firestore to send push notifications to users.

GoogleSignInService Class

This service centralizes all Google authentication logic and Firebase user handling.

Key Methods (Short)

- **initSignIn()**: Initializes Google Sign-In once using the Web Client ID.
- **signInWithGoogle()**: Authenticates with Google, signs in to Firebase, and creates a Firestore user document if it doesn't exist.
- **signOut()**: Signs out from both Google and Firebase.
- **getCurrentUser()**: Returns the currently authenticated Firebase user.

Initialization

```
static Future<void> initSignIn() async {  
  if (!isInitialize) {  
    await _googleSignIn.initialize(  
      serverClientId:  
        'YOUR_WEB_CLIENT_ID.apps.googleusercontent.com',  
    );  
  }  
  isInitialize = true;  
}
```

Purpose:

- Initializes Google Sign-In only once
- Uses Web Client ID for OAuth

`signInWithGoogle()` Method

Responsibilities

- Authenticate user with Google
- Exchange Google credentials with Firebase
- Save user profile in Firestore (if new user)

Code Explanation

```
static Future<UserCredential?> signInWithGoogle() async {  
  try {  
    initSignIn();  
  
    final GoogleSignInAccount googleUser = await _googleSignIn.authenticate();  
  
    final idToken = googleUser.authentication.idToken;  
  
    final authorizationClient = googleUser.authorizationClient;  
  
    GoogleSignInClientAuthorization? authorization =  
      await authorizationClient.authorizationForScopes(['email', 'profile']);  
  
    final accessToken = authorization?.accessToken;  
  
    if (accessToken == null) {
```

```

        throw FirebaseAuthException(code: "error", message: "Token error");
    }

    final credential = GoogleAuthProvider.credential(
        accessToken: accessToken,
        idToken: idToken,
    );

    final UserCredential userCredential =
        await FirebaseAuth.instance.signInWithCredential(credential);

    final User? user = userCredential.user;

    if (user != null) {
        final userDoc =
            FirebaseFirestore.instance.collection('users').doc(user.uid);

        final docSnapshot = await userDoc.get();

        if (!docSnapshot.exists) {
            await userDoc.set({
                'uid': user.uid,
                'name': user.displayName ?? '',
                'email': user.email ?? '',
                'photoURL': user.photoURL ?? '',
                'provider': 'google',
                'createdAt': FieldValue.serverTimestamp(),
            });
        }
    }

    return userCredential;
} catch (e) {
    print('Error: $e');
    rethrow;
}
}

```

Firestore User Schema

```

{
  "uid": "string",
  "name": "string",
  "email": "string",
  "photoURL": "string",
  "provider": "google",

```

```
"createdAt": "timestamp"
}
```

Sign Out

```
static Future<void> signOut() async {
  await _googleSignIn.signOut();
  await _auth.signOut();
}
```

Get Current User

```
static User? getCurrentUser() {
  return _auth.currentUser;
}
```

Complete Authentication Flow

1. User clicks **Google Sign-In** button
2. `_signinwithGoogle()` is triggered
3. Google authentication starts
4. Firebase authentication completes
5. User data stored in Firestore (if new)
6. Firebase ID Token retrieved
7. FCM token generated
8. User navigated to next screen

Best Practices

- Store FCM token in Firestore for push notifications
- Refresh FCM token on app restart
- Handle token expiration
- Secure Firestore rules

Ready for Production

This setup is production-ready and supports:

- Secure authentication
- Push notifications
- User session handling

Author: Flutter Firebase Integration  **Last Updated:** 2025